

Algorithm

1. Input.

2. Output.

3. Definiteness. - (ચાલકવ શરૂ કરે)

4. Finiteness. - (Appropriate for Te)

5. Effectiveness. - (Low time & space Complexity)

** વડે લોકાલા લોકાલ અન્ય "Quick Sort" વાલાશર કરા શુ
 "Time Limitation Exit" TLE ના શરૂઆ અન્ય.

Time Complexity $O(1)$ = Constant [સા રહેલો-રહેલો] $O(n)$ = Linear, $O(n^3)$ = Cubic, $O(\log n)$ = logarithmic. $O(n^2)$ = Quadratic. $O(2^n)$ = Exponential. $\rightarrow O(3^n) \rightarrow O(n^n)$ Order of Time Complexity

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 3^n < n^n$$

Asymptotic Notation

Time Complexity का Type Generate करते हैं।

There are Asymptotic Notation in 3 types.

① Big-O-Notation. $\rightarrow$ (Upperbound)

② Ω -Notation. $\rightarrow$ (Lowerbound)

③ Θ -Notation. $\rightarrow$ (Average)

* Time Complexity function $f(n)$.

* The order of Notation :-

$$[Big-O \rightarrow \Omega \rightarrow \Theta]$$

□ $f(n) = O(g(n))$ if $\exists +ve$, Constant value c .

$$\Rightarrow f(n) = c \times g(n) \quad [\forall n \geq 1]$$

Example: $f(n) = 2n + 3$

[Big-O-Notation এর জন্য
অবশ্যীয় $\leq$ অর্থাৎ $\leq$ বুলতে হবে
TC]

$$\Rightarrow f(n) = 2x + 3$$

$$\Rightarrow 2n + 3 \leq 2n + 3n$$

$$\Rightarrow 2n + 3 \leq 5n$$

[n চলে গেলে আর আরম্ভ যা থাকবে
তা দুই দফা Add করতে হবে]

$\therefore$ Time Complexity (TC) = $O(n)$.

$$1 < \log n < \sqrt{n} < \underbrace{n}_{\leftarrow X} < n \log n < n^2 < n^3 < 2^n < 3^n < n^n$$

$O(n)$ অবশ্যই আলাদা করে প্রমাণ করতে হবে, কিন্তু
 $\Theta(n)$ এর ক্ষেত্রে আলাদা করে প্রমাণ করতে হবে।

$\therefore f(n) = O(n) \rightarrow$ Upperbound.

☐ $[\geq] \rightarrow$ lowerbound এর জন্য প্রমাণ করতে হবে,
 $O(n), \Theta(n), \Omega(n)$

4th Formula :

- ① O-Notation: (Upperbound) $\rightarrow f(n) \leq g(n)$
② Ω -Notation: (Lowerbound) $\rightarrow f(n) \geq g(n)$

Example 5

4th $f(n) = 2n+3$ find Lowerbound Complexity.

$$\Rightarrow 2n+3 \leq 2n+3n \quad [\text{for upperbound}]$$

$$\Rightarrow 2n+3 \geq 2n \quad [\text{lowerbound}]$$

0 પિછુ હવાકાળા માય- હા- અમમ્મિ યાગુકાળુ-
value અમનિશ્ચ- વળ વગાળુ $f(n)$ નુ અમાન 3
વળુ અમાન પાગુમા માલુ.

$$\Rightarrow 2n+3 \geq 1.n$$

$$T.C = \Omega(n)$$

$$0 \quad \Omega(n) \leq f(n) \leq O(n)$$

↓
lowerbound

$$\Omega(n)$$

↓
upperbound
 $O(n)$

There are a common intersection is $\Theta(n)$.

So, $\Omega(n) = O(n)$

So, $\Theta(n)$. There are average T.C.

when, $\Omega(f(n)) \leq f(n) \leq O(n)$
 $\Omega(f(n)) \neq O(n)$

There are not Average Tc.

$$\square f(n) = 2n^2 + 3n + 4$$

For Upperbound,

$$\begin{aligned} f(n) &\leq g(n) \\ 2n^2 + 3n + 4 &\leq 2n^2 + 3n^2 + 4n^2 \\ 2n^2 + 3n + 4 &\leq 9n^2 \end{aligned}$$

$$\therefore \text{Upperbound (Big-O-Notation)} = O(n^2)$$

For lowerbound,

$$\begin{aligned} f(n) &\geq g(n) \\ 2n^2 + 3n + 4 &\geq n^2 \end{aligned}$$

$$\therefore \text{lowerbound} = \Omega(n^2)$$

For Average (Θ -Notation),

$$\Omega(n^2) \leq f(n) \leq O(n^2)$$

$$\Omega(n^2) \neq O(n^2)$$

$\therefore$ There are common intersection is (n^2)
 So There are average Tc exist, $\Theta(n^2)$.
 Q.E.D.

$$\boxed{f(n) = n^2 \log n + n}$$

For (Big-O - Notation),

$$f(n) \leq g(n)$$

$$n^2 \log n + n \leq n^2 \log n + n^2$$

We find, $O(n^2 \log n + n^2)$

$$= O(n^2 \log n) \text{ [simply by this]}$$

For (Ω - Notation),

$$f(n) \geq g(n)$$

$$n^2 \log n + n \geq n^2 \log n$$

$$\Omega(n^2 \log n)$$

For (Θ - Notation),

$$\Omega(n^2 \log n) \leq f(n) \leq O(n^2 \log n)$$

$$\Omega(n^2 \log n) = O(n^2 \log n)$$

There are common intersection is $(n^2 \log n)$
so, Θ - Notation = $\Theta(n^2 \log n)$

$$\boxed{\text{Q}} \quad f(n) = n!$$

$$f(n) = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$$

$$f(n) \leq g(n)$$

$$n! \leq n^n \quad [n \times n \times n \times \dots]$$

$$\text{Big-O-Notation} = O(n^n)$$

For lower bound,

$$f(n) \geq g(n)$$

$$n! \geq 1$$

$$\Omega\text{-Notation} = \Omega(1)$$

Average ,

$$\Omega(1) \leq f(n) \leq O(n^n)$$

$$\Omega(1) \neq O(n^n)$$

There are not common intersection,

So, Average Time complexity or Θ -Notation does not exist.

$$\boxed{\square} \quad f(n) = \log n!$$

For (Big-O-Notation),

$$f(n) \leq g(n)$$

$$\log n! \leq \log (n \times n \times n \times \dots \times n)$$

$$\log n! \leq \log n^n$$

$$\log n! \leq n \log n. \quad \therefore O = O(n \log n)$$

For (Ω -Notation),

$$f(n) \geq g(n)$$

$$\log n! \geq \log(1)$$

$$\log n! \geq 1 \quad \therefore \Omega = \Omega(1)$$

For (Θ -Notation),

$$\Omega(1) \leq f(n) \leq O(n \log n)$$

$$\Omega(1) \neq O(n \log n)$$

So There are not common intersection.
So, Average Tc is not exist.

Properties of Asymptotic Notation.

* General Property: if $f(n) = O(g(n))$ then
 $a * f(n) = O(g(n))$

$$f(n) = 2n^2 + 5 = O(n^2)$$

$$7 * f(n) = (2n^2 + 5) \times 7 = 14n^2 + 35 = O(n^2)$$

* Reflexive Property:

if $f(n)$ is given that,

$$O(f(n)) = f(n)$$

$$g(n) = f(n)$$

$$f(n) = n$$

* Transitive Property:

if $f(n)$ is $O(g(n))$

$$g(n) = O(h(n))$$

$$f(n) = O(h(n))$$

$$\boxed{\text{Ex}} \quad f(n) = n; \quad g(n) = n^2$$

$$h(n) = n^3; \quad f(n) = O(g(n))$$

$$f(n) = O(n^3)$$

$$O = \Omega = \Theta.$$

এবার অন্য চেষ্টা।

Symmetric Property: $\log_a b$ and $\log_b a$ are reciprocals of each other, $\log_a a = 1$ and $\log_a 1 = 0$.

Transpose Symmetric Property: $\log_a b = \frac{1}{\log_b a}$.

Compare of 2 function

a

b

n^2

n^3

Apply (log) $\Rightarrow$

$$1. \log(ab) = \log a + \log b.$$

$$2. \log\left(\frac{a}{b}\right) = \log a - \log b.$$

$$3. \log a^b = b \log a.$$

$$4. a \log e^b = b \log e^a.$$

Ex

n^2

$\log n^2$

n^3

$\log n^3$

$2 \log n$

$3 \log n$

$2 \log n < 3 \log n$

$[n^2 < n^3]$

[যখন log Apply করা হয়
তখন যেখানে কোটা
দ্বিগুন থাকবে তাকে
বাই বলা হবে না]

$$\boxed{12} \quad 2 \log n \quad 3 \log n$$

By eliminating Constant,

$$2 \log n \quad 3 \log n$$

$$\text{or, } \log n \quad \log n$$

$$\text{or, } f(n) \quad g(n).$$

$$\boxed{13} \quad f(n) = n^2 \log n \quad g(n) = n (\log n)^{10}$$

By Applying log we can get,

$$\log (n^2 \log n)$$

$$\log (n (\log n)^{10})$$

$$\text{or, } \log n^2 + \log \log n$$

$$\log n + \log (\log n)^{10}$$

$$\text{or, } 2 \log n + \log \log n$$

$$\log n + 10 \log \log n$$

$$\text{or, } 2 \log n + \log \log n > \log n + 10 \log \log n$$

$$\therefore f(n) > g(n)$$

$$\boxed{\text{Q}} \quad f(n) = 3n^{\sqrt{n}}$$

$$g(n) = 2^{\sqrt{n}} \log n$$

$$\text{or, } 3n^{\sqrt{n}}$$

$$2^{\sqrt{n}} \log n$$

$$\text{or, } 3n^{\sqrt{n}}$$

$$2 \log n^{\frac{1}{2}}$$

$$[a \log e^b = b \log e^a]$$

$$\text{or, } 3n^{\sqrt{n}}$$

$$(n^{\sqrt{n}}) \log_2^2$$

[অন্য বিধি-
অন্য বিধি log
নলে]

$$\therefore 3n^{\sqrt{n}}$$

$$n^{\sqrt{n}}$$

$$\boxed{\text{Q}} \quad f(n) = n \log(n)$$

$$g(n) = 2^{\sqrt{n}}$$

Applying log in both side,

$$\text{or, } \log n \log(n)$$

$$\log 2^{\sqrt{n}}$$

$$\text{or, } \log(n) \times \log(n)$$

$$\sqrt{n} \log 2$$

$$\text{or, } \log^2 n$$

$$\sqrt{n} \log 2$$

$$\therefore \log^2 n < \sqrt{n}$$

Recurrence equation, Solution

Best	Worst	Average
↓	↓	↓
4051	64/201	515
↓	↓	
Constant	Functional	
↓	↓	
$O(1)$	$O(n)$	
	$O(n^2)$	
	$O(n)$	

Best Case $\rightarrow O(1)$

Worst Case $\rightarrow O(n)$

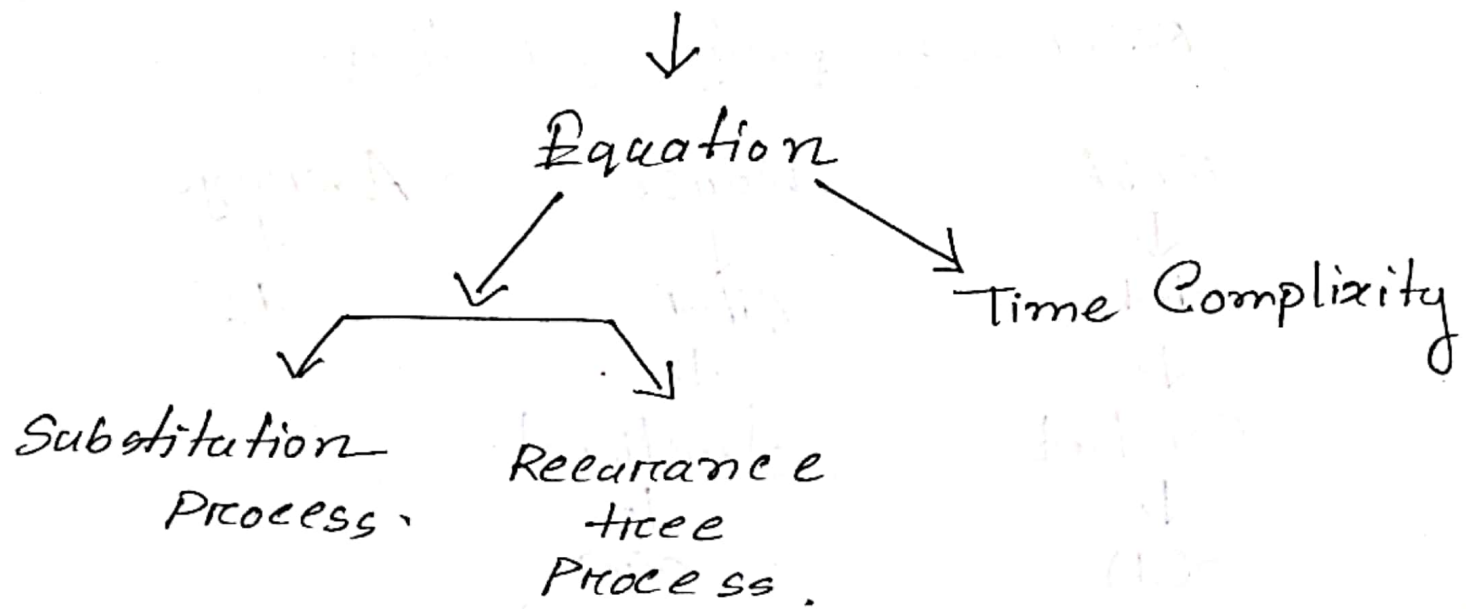
Divide and Conquer

↓
approach for Solving Problem.

Uses of Divide & Conquer:

1. Binary Search.
2. Finding Max and Min value.
3. Merge Sort.
4. Quick Sort.
5. Matrix Multiplication.

Recurrence Solution



Example:

```
Void test (int n) {  
    if (n > 0) {  
        Print (n);      → 1  
        test (n-1);      → T(n-1)  
    }  
}
```

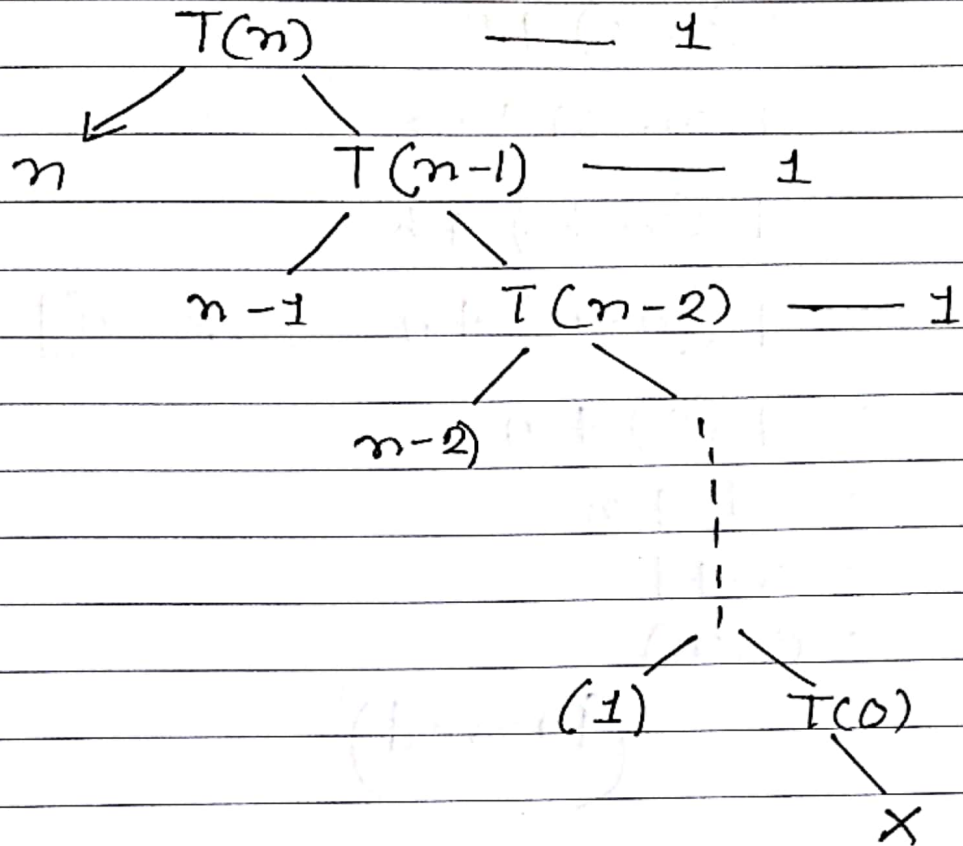
$$T(n) = T(n-1) + 1 \quad n > 0$$

Recurrence Equation:

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1; & n > 0 \end{cases}$$

[अब किंदूव Return type by default 1 इस समझ $n=0$]

Recurrence Tree:



$$\begin{aligned} T(n) &= 1 + 1 + 1 + \dots + 1 \\ &= n \times 1 \\ &= O(n) \end{aligned}$$

Substitution Method:

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + 1 & n>0 \end{cases}$$

$$T(n) = T(n-1) + 1$$

$$= [T(n-2) + 1] + 1$$

$$= T(n-2) + 2$$

$$= T(n-3) + 3$$

$$= T(n-k) + k$$

$$= T(n-n) + n \quad [k=n]$$

$$= T(0) + n$$

$$= 1 + n$$

$$= n+1$$

$$= O(n)$$

(Proved)

Example-2 :

```

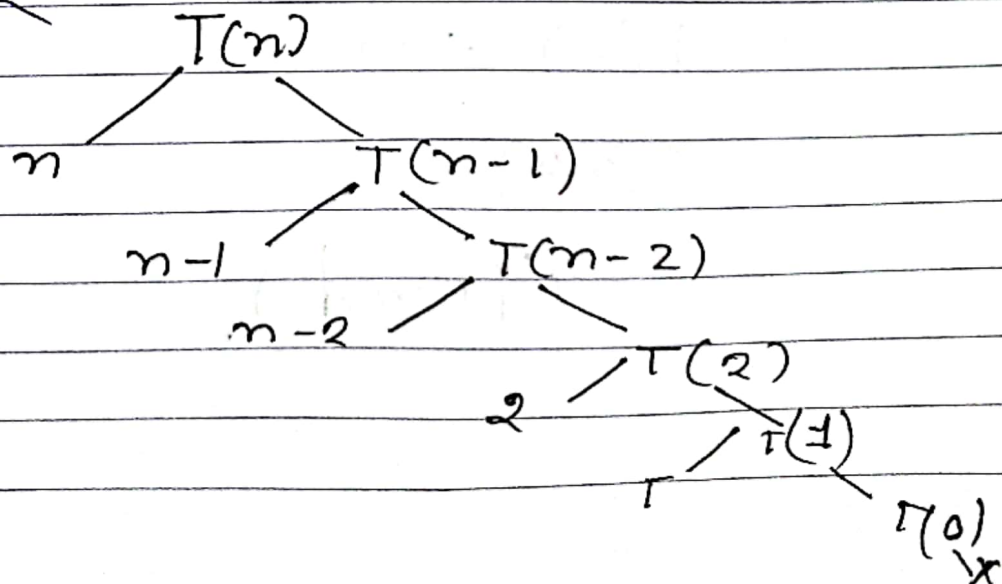
Void test (int n) {
    if (n > 0) {
        for (i = 0; i < n; i++)
        {
            Print { n };    — n
        }
        test (n-1);    — n-1
    }
}

```

$$T(n) = T(n-1) + n$$

Recurrence Equation :

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + n & n>0 \end{cases}$$

Recurrence Tree :

$$\begin{aligned}
 T(n) &= 1 + 2 + 3 + \dots + (n-2) + (n-1) + \dots + n \\
 &= \frac{n(n+1)}{2} \\
 &= \frac{n^2 + n}{2} \\
 &= n^2 \\
 &= O(n^2)
 \end{aligned}$$

Recurrence Subtraction:

$$T(n) = T(n-1) + n$$

$$= [T(n-2) + n] + n$$

$$= T(n-2) + 2n$$

$$= T(n-3) + 3n$$

$$= \begin{array}{c} T(n-k) \\ \vdots \\ T(0) \end{array} \quad \begin{array}{c} | \\ | \\ | \end{array}$$

$$= T(n-k) + kn \quad [k=n]$$

$$= T(n-n) + n^2$$

$$= T(0) + n^2$$

$$= O(n^2)$$

(Proved).

Code Edit: Select

Master Method (for decreasing function)

Decreasing function test ($n-1$)

Divide function test ($n/2$)

* If function satisfies both test then it is a Decreasing function.

* If function satisfies both test then it is a Divide function.

Decreasing Function

$$T(n) = aT(n-b) + f(n) \quad a > 0, b > 0 \text{ and } f(n) = O(n^k); k \geq 0.$$

When $a=1, b=1$ then $O(n \times f(n))$.

$$1. T(n) = T(n-1) + 1 \Rightarrow O(n)$$

$$2. T(n) = T(n-1) + n \Rightarrow O(n^2)$$

$$3. T(n) = T(n-1) + \log n \Rightarrow O(n \log n)$$

$$O(n \times f(n))$$

☐ when, $a > 1$ then $O(a^{\frac{n}{b}} \times f(n))$

1. $T(n) = 2T(n-1) + 1 \Rightarrow O(2^n)$

2. $T(n) = 3T(n-1) + 1 \Rightarrow O(3^n)$

3. $T(n) = 2T(n-1) + n \Rightarrow O(n2^n)$

4. $T(n) = 3T(n-1) + n \Rightarrow O(n3^n)$

☐ if $a < 1$ then $O(f(n))$

Ex: $T(n) = 2T(n-1) + 1$

By using Master Method,

we know that, when $a > 1$

where,

$$a = 2$$

$$b = 1$$

$$\text{Then } T(n) = O(a^{\frac{n}{b}} \times f(n))$$

$$= O(2^{\frac{n}{1}} \times 1)$$

$$= O(2^n)$$

(Ans)

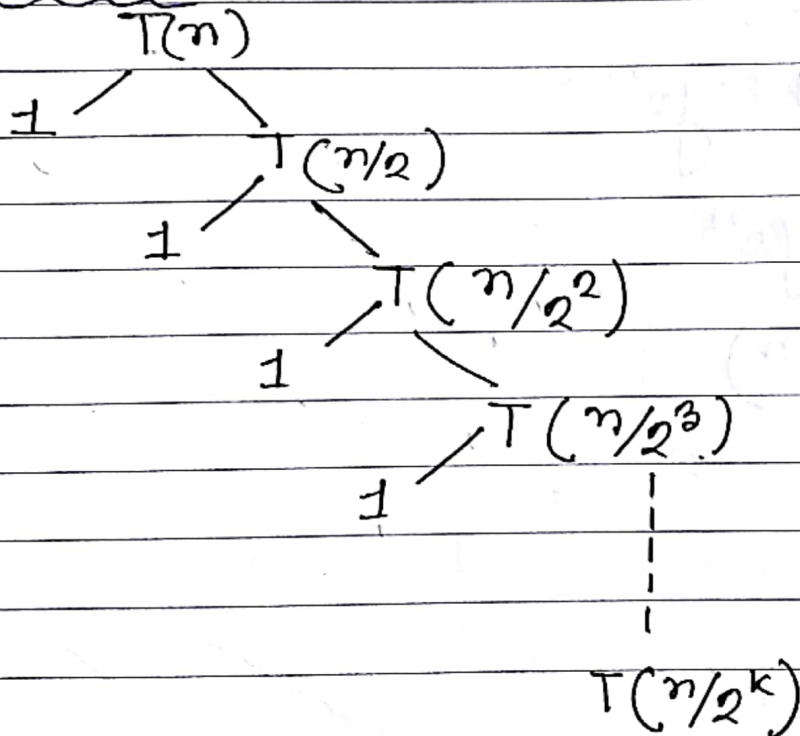
Dividene Function

```

test (int n) {
    if (n > 1)
    {
        Print (n);
        test (n/2);
    }
}

```

Recurrence tree:



$$\begin{cases} T(n/2) + 1 & n > 1 \\ 1 & n = 1 \end{cases}$$

$$\frac{n}{2^k} = 1;$$

$$n = 2^k;$$

$$k = \log_2 n.$$

$$\begin{aligned} T(n) &= 1 + 1 + 1 + \dots + 1 \\ &= k \times 1 \\ &= 1 \times \log_2 n \end{aligned}$$

Substitution Method

$$T = T(n/2) + 1$$

$$= T(n/2^2) + 1 + 1$$

$$= T(n/2^3) + 3$$

$$\vdots$$
$$= T\left(\frac{n}{2^k}\right) + k$$

$$= T(1) + \log_2 n$$

$$= 1 + \log_2 n$$

$$= O(\log_2 n)$$

$\Rightarrow$

Master's Method for Divide and Conquer

$$T(n) = aT(n/b) + f(n)$$

$$a \geq 1 \quad f(n) = O(n^k \log n^p)$$

$$b > 1$$

$$\text{if } \log_b a > k \quad O(n \log_b a)$$

$$\text{if } \log_b a = k$$

$$p > -1 \quad O(n^k \log^{p+1} n)$$

$$p = -1 \quad O(n^k \log \log n)$$

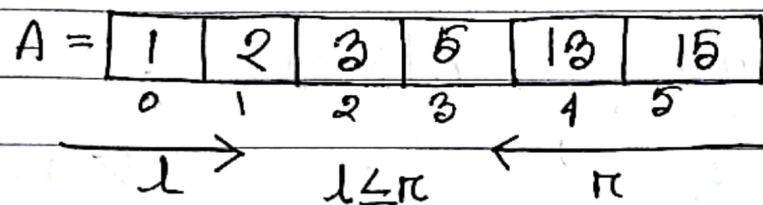
$$p < -1 \quad O(n^k)$$

$$\text{if } \log_b a < k \quad \text{if } p \geq 0 \quad O(n^k \log^p n)$$

$$p < 0 \quad O(n^k)$$

Iterative Method (using loop)

Binary Search



```

Code: l, r;
      flag = 0;
      while (l ≤ r)
      {
          mid = (l + r) / 2;
          if (key == A[mid])
          {
              flag = 1;
              print (mid);
              break;
          }
          else if (key < A[mid])
          {
              r = mid - 1;
          }
          else if (key > A[mid])
          {
              l = mid + 1;
          }
      }
  
```

Binary Search 2nd step — nlogn

A =

1	2	3	10	13	15
---	---	---	----	----	----

 where key = 10
 0 1 2 3 4 5

l	r	mid	A[mid]	
0	5	2	3	k=10
3	5	4	13	
3	3	3	10	k = A[mid] break;

By using Binary Search we find $k = A[mid]$
 $k = 10$.

A =

1	2	3	10	13	15
---	---	---	----	----	----

 where key = 100

l	r	mid	A[mid]	
0	5	2	3	k=100
3	5	4	13	
5	5	5	15	
6	5	X	X	— $l > r$ break;

Recursive $T(n)$

```

binary_Search (int l, int r, key)
{

```

```

    mid = (l + r) / 2;

```

```

    if (l > r)

```

```

        return -1;

```

```

    if (key == A[mid]) {
        return mid;
    }

```

```

    else if (key < A[mid]) {

```

```

        r = mid - 1;
    }

```

```

    else if (key > A[mid]) {

```

```

        l = mid + 1;

```

```

        l = mid + 1;
    }

```

```

    binary_Search (l, r, key)
}

```

$$[T(n) = T(n/2) + 1]$$

Radix Sort

1. Non Comparison based Algo.
2. Base Based Algorithm.
3. Sorting process starts by using base comparison. it starts from the LSB (Least significant Bit).

	904	46	5	74	62	1
	↓	↓		↓		
904	001	001		001		001
046	062			904		005
005	904			005		046
074	074			046		062
062	005			062		074
001	046			072		901

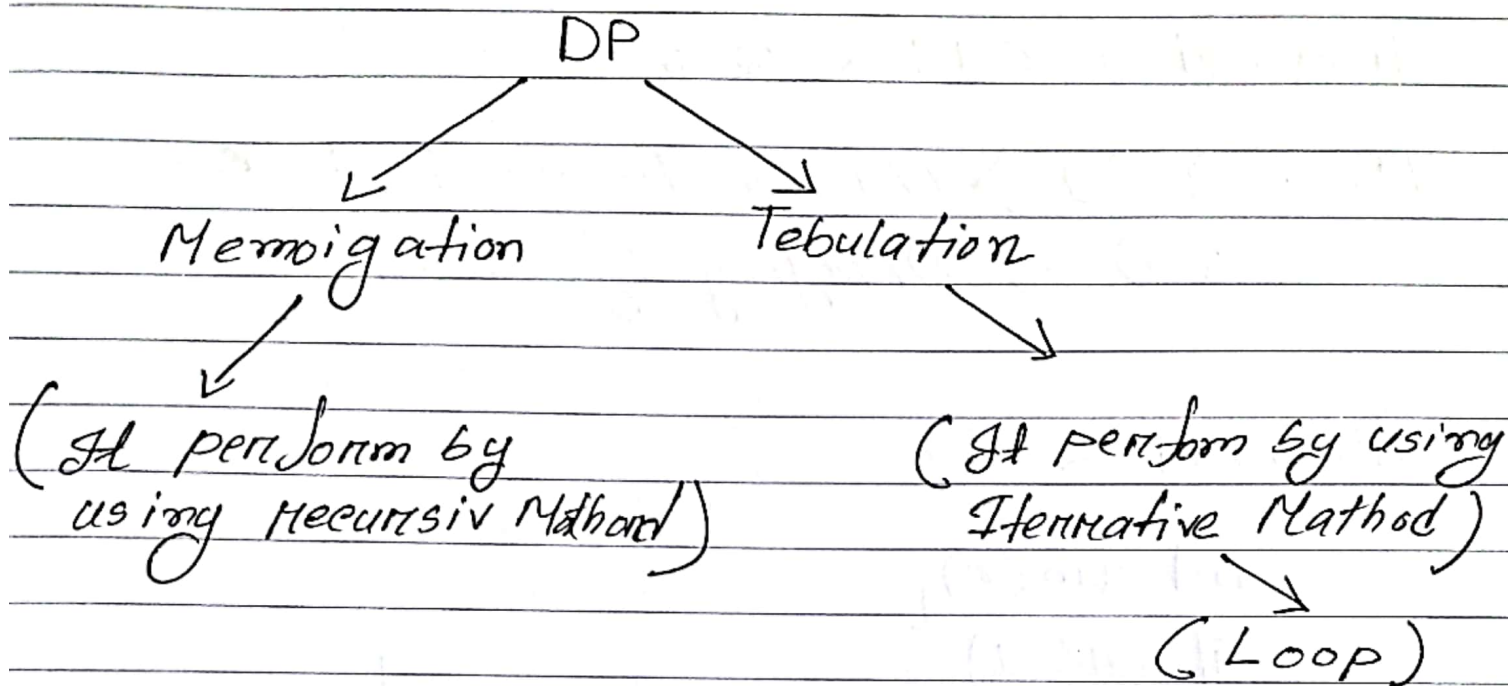
→ → →

Time Complexity = $O[n^2]$.

$$= 3 \times 6$$

$$= 18$$

Dynamic Programming



□ Problem Optimize or Optimization problem (output minimum or maximum result) এর ক্ষেত্রে অন্য Dynamic Programming দ্বারা করা হয়।

□ Test case ছোট হলে Greedy সহ, test case বড় হলে DP দ্বারা করা হয়।

□ Dynamic Programming → (Consuming TC)

□ Example: Fibonacci = 0 1 1 2 3 5 8 ... n
 এর ক্ষেত্রে 100 তম সংখ্যা বের করতে চাইলে loop use করা হয় তবে Greedy Apply করে সমাধান করা যায়।

ଯଦି 1000 ଓ 1000 ସଂଖ୍ୟା ଉପର କରାଯିବ ବଳାୟିତ ଓଡ଼ିଆ
Dynamic Programming ବ୍ୟବହାର କରାଯିବ।

Fibonacci = 0 1 1 2 3 5 n

TC = $O(2^n)$ $\Rightarrow$ Normal Fibonacci process.
= $O(n)$ $\Rightarrow$ Applying Dynamic process.

Fibonacci =

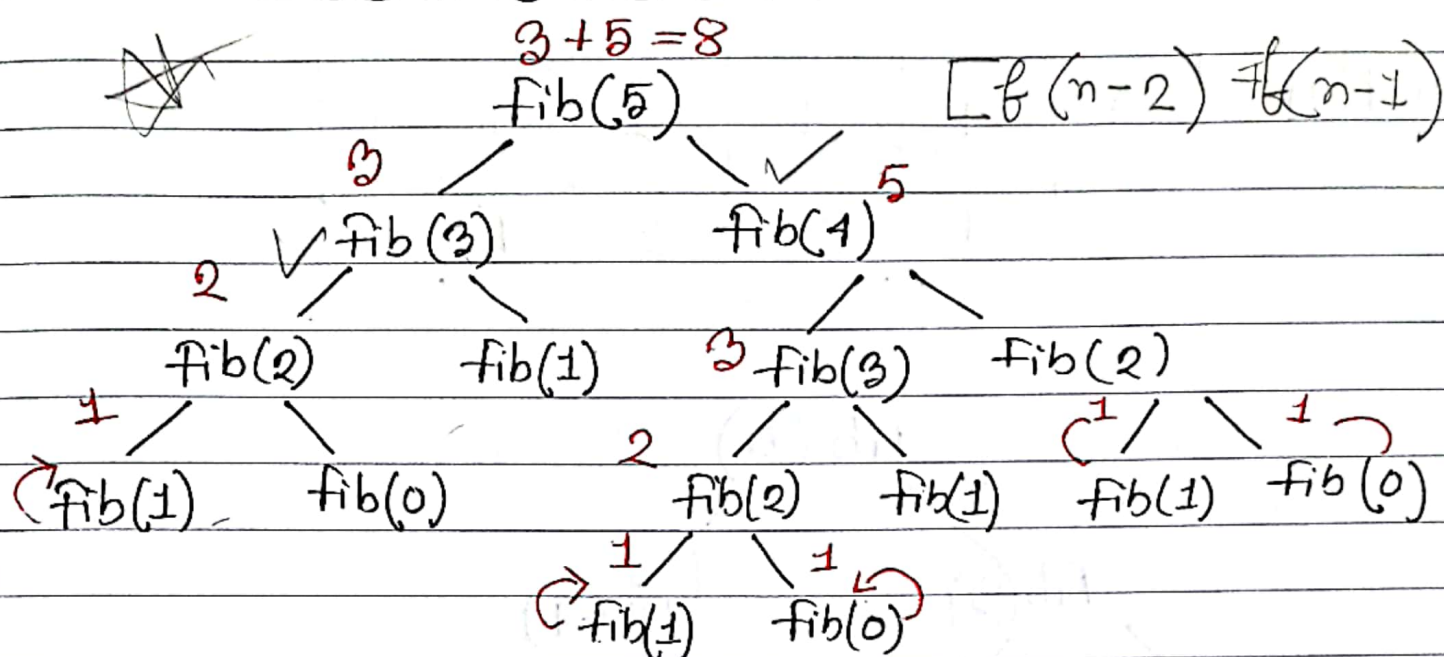
```
int fib(n){  
    if (n <= 1) ----- 1  
        return 1;
```

```
    return fib(n-2) + fib(n-1); ----- T(n-1) T(n-2)  
}
```

[Decreasing function]

□ $(n-1)$ = Decreasing function.

□ $n/2$ = Dividing function.

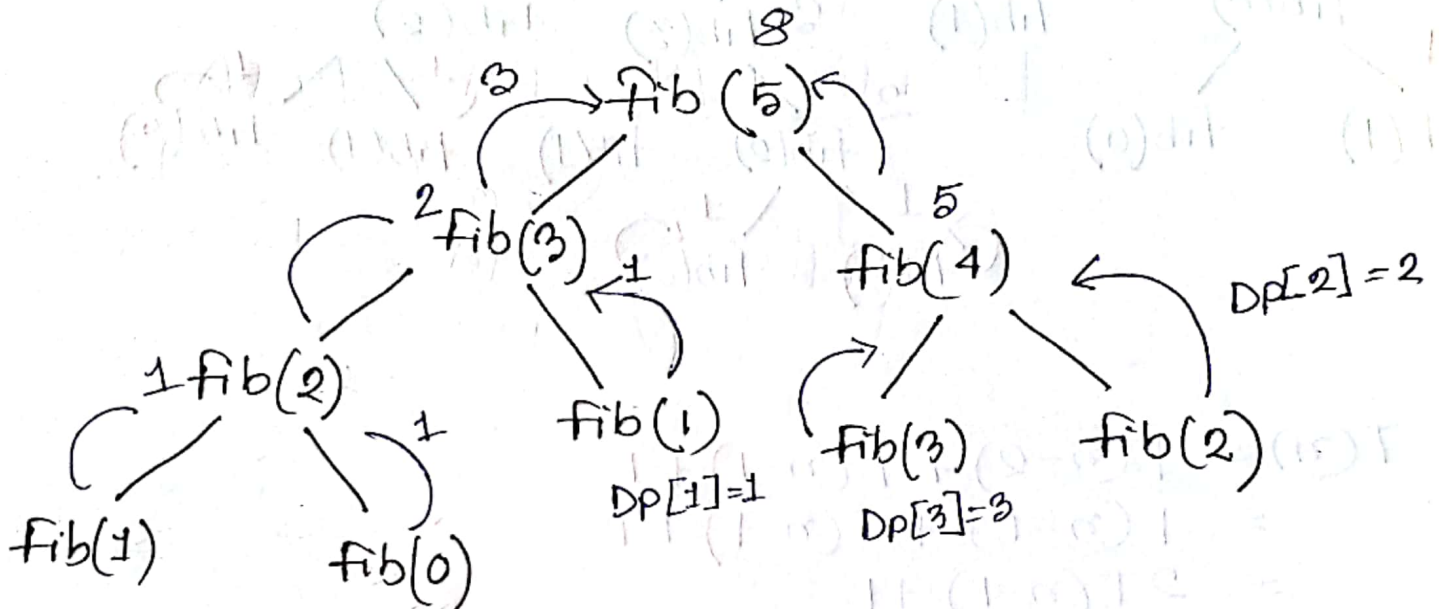
Recursive Tree

$$\begin{aligned}
 T(n) &= T(n-2) + T(n-1) + 1 \\
 &= T(n-1) + T(n-1) + 1 \\
 &= 2T(n-1) + 1 \\
 &= O(2^n)
 \end{aligned}$$

Dynamic Programming

$DP[n] =$

-1	-1	-1	-1	-1	-1
0	1	2	3	4	5



$DP[n]$

Stored Array:

0	1	2	3	5	8
0	1	2	3	4	5

$$= O(n)$$

Code:

```
int fib(n) {
```

```
    if (n <= 1)
```

```
        return 1;    [Base Case]
```

```
    if (A[n] == -1)
```

```
    {
```

```
        num = fib(n-2) + fib(n-1);    [Recursive Call]
```

```
        A[n] = num;
```

```
        return num;
```

```
    }
```

```
else return A[n];
```

```
}
```

Tebulation (^{iterative} ~~recursive~~ Method) - (loop)

$A[0] = 0;$

$A[1] = 1;$

for ($i=2; i < n; i++$) {

$A[i] = A[i-1] + A[i-2];$

}

cout << $A[n]$ << endl;

Matrix chain Multiplication

Using Dynamic Programming.

$$A = \begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix}$$

$$= 2 \times 2$$

$$B = \begin{bmatrix} a_2 & b_2 \\ c_2 & d_2 \end{bmatrix}$$

$$= 2 \times 2$$

$$A \times B = \begin{bmatrix} a_1 a_2 + b_1 c_2 & a_1 b_2 + b_1 d_2 \\ a_2 c_1 + c_2 d_1 & b_2 c_1 + a_1 d_2 \end{bmatrix}$$

$$A \times B = C$$

where the dimension = $(3 \times 4)(4 \times 3)$ valid
 $= (3 \times 3)$,

□ Row Column ના ફક્ત બેર ના Common Point ના ફક્ત દ્વારા શકે Matrix Multiplication કરાયા,

$$\begin{aligned} & (2 \times 2) (2 \times 2) \\ & = 2 \times 2 \times 2 \\ & = 8 \end{aligned}$$

$$\begin{aligned} & (4 \times 3) (3 \times 10) \\ & = (4 \times 10 \times 3) \end{aligned}$$

$$\boxed{A_1 \times A_2 \times A_3}$$

$$(2:3) \quad (3:4) \quad (4:2)$$

$$\Rightarrow \cancel{(A_1 \times A_2)} \times A_3$$

$$\Rightarrow \cancel{(2 \times 3 \times 4)} \times A_3$$

$$\Rightarrow \cancel{24} \times$$

$$\Rightarrow (A_1 \times A_2) \times A_3$$

$$\text{first term, } (A_1 \times A_2)$$

$$= (2:3) \times (3:4)$$

$$= 2 \times 3 \times 4$$

$$= 24$$

$$\text{2nd term} = (A_1 \times A_2) \times A_3$$

$$= \{(2:3)(3:4)\} \times (4:2)$$

$$= (2:4) \times (4:2)$$

$$= \cancel{(2:2)} \quad 2 \times 2 \times 4$$

$$= 16$$

$$\text{full term, } = 24 + 16$$

$$= 40.$$

$$\boxed{A_1 \times (A_2 \times A_3)}$$

$$= (2:3) (3:4) (4:2)$$

$$\Rightarrow (A_1 \times A_2) \times A_3 \quad A_1 \times (A_2 \times A_3)$$

$$\begin{aligned} \text{first term} &= (A_2 \times A_3) \\ &= (3:4) (4:2) \\ &= (3 \times 2 \times 4) \\ &= 24 \end{aligned}$$

$$\text{Second term} = A_1 \times (A_2 \times A_3)$$

$$\begin{aligned} &= (2:3) \times \{(3:4) (4:2)\} \\ &= (2:3) \times (3:2) \\ &= (2 \times 2 \times 3) \\ &= 12 \end{aligned}$$

$$\begin{aligned} \text{Full term} &= (24 + 12) \\ &= 36 \end{aligned}$$

Matrix Multiplication Chain using DP

A_1	A_2	A_3	A_4
$(3:2)$	$(2:4)$	$(4:2)$	$(2:5)$
$d_0 d_1$	$d_1 d_2$	$d_2 d_3$	$d_3 d_4$

When $j-i=1$
 $j-i=2$
 $j-i=3$

$$c[1,2] = \min_{1 \leq k < 2} \left\{ \begin{array}{l} c[i, k] + c[k, j] + d_0 \times d_1 \times d_2 \end{array} \right.$$

then $c[1,2] = \min_{1 \leq k < 2} \left\{ \begin{array}{l} k=1; c[1,1] + c[2,2] + d_0 \times d_1 \times d_2 \\ k=1; 0 + 0 + 3 \times 2 \times 4 = 24 \end{array} \right.$

$c[2,3] = \min_{2 \leq k < 3} \left\{ \begin{array}{l} k=2; c[2,2] + c[3,3] + d_1 \times d_2 \times d_3 \\ k=2; 0 + 0 + 2 \times 4 \times 2 = 16 \end{array} \right.$

$c[3,4] = \min_{3 \leq k < 4} \left\{ \begin{array}{l} k=3; c[3,3] + c[4,4] + d_2 \times d_3 \times d_4 \\ k=3; 0 + 0 + 4 \times 2 \times 5 = 40 \end{array} \right.$

$c[1,3] = \min_{1 \leq k < 3} \left\{ \begin{array}{l} k=1; c[1,1] + c[2,3] + d_0 \times d_1 \times d_3 \\ k=1; 0 + 16 + 3 \times 2 \times 2 = 28 \end{array} \right.$

Figure No.

$$c[1,3] = \min_{1 \leq k \leq 3} \begin{cases} k=2; c[1,2] + c[3,3] + d_0 \times d_2 \times d_3 \\ k=2; 24 + 0 + (3 \times 4 \times 2) = 48 \end{cases}$$

$$c[2,4] = \min_{2 \leq k \leq 4} \begin{cases} k=2; c[2,2] + c[3,4] + d_1 \times d_2 \times d_4 \\ k=2; 0 + 40 + (2 \times 4 \times 5) = 80 \\ k=3; c[2,3] + c[4,4] + d_1 \times d_3 \times d_4 \\ k=3; 16 + 0 + 2 \times 2 \times 5 = 36 \end{cases}$$

$$c[1,4] = \min_{1 \leq k \leq 4} \begin{cases} k=1; c[1,1] + c[2,4] + d_0 \times d_1 \times d_4 \\ k=1; 0 + 36 + 3 \times 2 \times 5 = 54 \\ k=2; c[1,2] + c[3,4] + d_0 \times d_2 \times d_4 \\ k=2; = 24 + 40 + (3 \times 2 \times 5) = 71 \\ k=3; c[1,3] + c[4,4] + d_0 \times d_3 \times d_4 \\ k=3; = 28 + 0 + 3 \times 2 \times 5 = 58 \end{cases}$$

C

	1	2	3	4
1	0	24	28	(52)
2		0	16	36
3			0	24
4				0

K

	1	2	3	4	
1	0	(1	1	3)	$(A_1 \times A_2 \times A_3)$
2		0	(2	3)	$(A_2 \times A_3)$
3			0	3	A_4
4				0	

$$\begin{aligned}
 & A_1 \quad A_2 \quad A_3 \quad A_4 \\
 & (A_1 \quad A_2 \quad A_3) \times A_4 \\
 & ((A_1 \times (A_2 \times A_3)) \times A_4)
 \end{aligned}$$

LCS (Longest Common Subsequence.)

String 1: $[a\ b\ c\ d\ e\ f\ g\ h\ i\ j]$ - Parents

String 2: $[e\ c\ d\ g\ i]$ - child

-ଏହା Sub Sequence ସେବା କରାଯାଏ ବୋଲି ।

* Sub String: Sequence ଅନ୍ତର୍ଗତ କିଛି ଅଂଶ

* Sub Sequence: Sequence Order ରୁ ଅନ୍ତର୍ଗତ ଅଂଶ

st1 = a b c d e f g h i

st2 = e c d g i

largest = e g i

$[e\ c\ d\ g\ i] = 4$ [largest]

ii) Dynamic Programming आसकसु Top Down Method
- follow करु, $DP[n][m]$ full fill कराव जेस ।

There are Two Kind of LCS,

① Memoization $\rightarrow$ 2D DP Array या निह एक filler
रुम आसुव पुरु भासु, याकर
(Bottom Up) Method वल ।
 $TE = O(n \times m)$.

ii) Dynamic Programming $\rightarrow$ (Top Down) $\rightarrow$ 6 2D Array
आस एक- fill रुम निह आसुव
 $TE = O(n \times m)$
 $\downarrow$
(Size + Subsequence)

* Memoization

prev year

(' \0' = Newline ;

if $Les(i, j) \{$

if $(A[i] == '\0' \parallel B[j] == '\0')$

return 0;

else if $(A[i] == B[j])$

$\{ return (1 + Les(i, j+1)) \}$

else max $\{ Les(i+1, j), Les(i, j+1) \}$

A =

b	d	\0
---	---	----

 — n

LCS [m][n]

B =

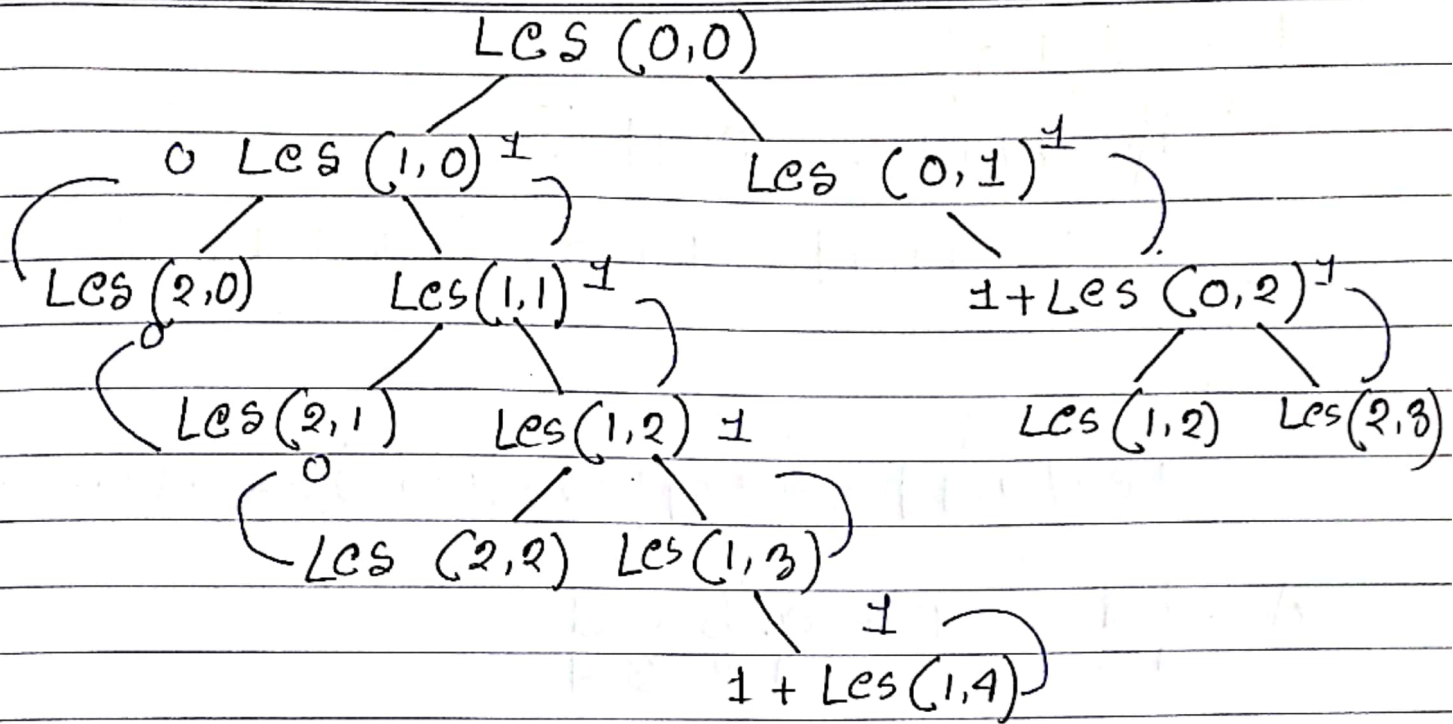
a	b	e	d	\0
---	---	---	---	----

 — m

	a	b	e	d	\0
0	2	2	1		
b 1	1	1	1	1	0
d 2	0	0	0	0	

Bottom up

LCS = [n x m]
base = [0, 0]



$$LCS = [m \times n]$$

$$TC = O(n \times m)$$

$$\text{Worst case} = O(n^2).$$

Dynamic Solution

~~Les~~ (i, j) {

if (A[i] == B[j]) {

Les [i, j] = 1 + Les (i-1, j-1)

}

else

Les [i, j] = max [Les [i-1, j], Les [i, j-1]]

A = b d
1 2

B = a b e d
1 2 3 4

		a	b	c	d	
		0	1	2	3	4
0		0	0	0	0	0
b	1	0	0	1	1	1
d	2	0	0	1	1	(2)

↓

Top
Down

Size.

LES = [2, 4]

$$\begin{aligned} Lcs = [1, 1] &= \max(Lcs[0, 1], Lcs[1, 0]) \\ &= \max(0, 0) \\ &= 0 \end{aligned}$$

$$\begin{aligned} Lcs = [1, 2] &= \max 1 + Lcs[0, 1] \\ &= 1 + 0 \\ &= 1 \end{aligned}$$

$$Lcs = [1, 3] = \dots\dots\dots$$

$$Lcs = [2, 4] = \dots\dots\dots$$

□ Column આગળ Row ત્રિજાનું લેખનાંકનિ આગળ કરવું હશે.

□ || || ના ત્રિજાનું વધુ વધારું હશે.

		a	b	c	d
	0	1	2	3	4
b 0	0	0	0	0	0
b 1	0	0	1	1	1
d 2	0	0	1	1	2

largest common Subsequence = bd.

এই সারণী থেকে কোনােকানি উপায় দিতে বা Count
হয়।

Counting Sort (Non-Comparisomal Sort)

$$A = [2, 1, 2, 3, 1, 2, 4]$$

where $n = \text{array size}$

$k = \text{Numbers of range}$

Condition $0 \leq x \leq k$

Negative value x

↓
Counting
↓
Radix
↓
Bucket

Here $n=7$, $k=5$

$$A = [2, 1, 2, 3, 1, 2, 4]$$

$$\text{Count Array} = \begin{array}{|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & 2 & 3 & 4 & 5 \\ \hline \end{array}$$

$$= \begin{array}{|c|c|c|c|c|c|} \hline 0 & 2 & 3 & 1 & 1 & 0 \\ \hline 0 & 1 & 2 & 3 & 4 & 5 \\ \hline \end{array}$$

$$\text{Count}[0] = 0$$

$$\text{Count}[1] = 2 = 11$$

$$\text{Count}[2] = 3 = 11222$$

$$\text{Count}[3] = 1 = 112223$$

$$\text{Count}[4] = 1 = 1122234$$

$$\text{Counting Sort} = 1122234$$